

# Assignment 2

## Wireframe Viewer and Geometric Transformations

Computer Graphics Course  
C++ | CLion | MiniFB | MicroUI | GLM

|                   |                                                |
|-------------------|------------------------------------------------|
| <b>Student 1</b>  | Natalie Bittar ID:212586044                    |
| <b>Student 2</b>  | Adan Awad ID:324302678                         |
| <b>Course</b>     | Computer Graphics                              |
| <b>Assignment</b> | Wireframe Viewer and Geometric Transformations |

*This report documents the design, implementation, testing, and results of the 3D wireframe viewer developed for Assignment 2.*

## Report Structure

- Part 0: GLM Integration
- Part 1: Loading and Inspecting 3D Data
- Part 2: Normalization and Viewport Transformation
- Part 3: Orthographic Projection and Wireframe Rendering
- Part 4: Transformation Matrices and Immediate Mode GUI
- Part 5: Applying Local and World Transformations
- Part 6: Interactive Input Modifiers
- Part 7: Pair Programming Extensions
- Conclusion

## Part 0: GLM Integration

GLM was integrated through CMake using FetchContent and linked to the minigui target. GLM provides vectors, matrices, and transformation utilities that are compatible with graphics-oriented conventions.

```
FetchContent_Declare(
    glm
    GIT_REPOSITORY https://github.com/g-truc/glm.git
    GIT_TAG 1.0.1
)
FetchContent_MakeAvailable(glm)

target_link_libraries(minigui PRIVATE microui_lib minifb glm::glm)
```

The project configured successfully and reported GLM version 1.0.1. The code uses glm::vec3, glm::vec4, glm::mat4, glm::translate, glm::rotate, and glm::scale.

## Part 1: Loading and Inspecting 3D Data

A custom OBJ loader was implemented. It reads vertex records beginning with v and face records beginning with f. Positive and negative OBJ indices are supported, and polygonal faces are triangulated using a triangle fan.

```
struct Face {
    int a;
    int b;
    int c;
};

struct Mesh {
    std::vector<glm::vec3> vertices;
    std::vector<Face> faces;
};
```

Three small models are created and loaded: a cube, a pyramid, and an octahedron. The GUI displays the number of vertices and faces of the selected model. For example, the cube contains 8 vertices and 12 triangular faces.

## Part 2: Normalization and the Viewport Transform

OBJ coordinates may have arbitrary sizes and positions. To make every model visible, the minimum and maximum coordinates are computed independently for x, y, and z. These values define the axis-aligned bounding box.

```
mesh.center = (mesh.minimum + mesh.maximum) * 0.5f;
glm::vec3 size = mesh.maximum - mesh.minimum;
float largest_dimension = std::max({size.x, size.y, size.z});
mesh.normalization_scale = 2.0f / largest_dimension;
```

The center is translated to the origin, and a uniform scale factor maps the largest model dimension to approximately the range [-1, 1]. Uniform scaling preserves the original proportions and avoids distortion.

Mathematically, for a vertex  $v$ , the normalized position is:

$$v_{\text{normalized}} = s * (v - \text{center})$$

where  $s = 2 / \max(\text{width}, \text{height}, \text{depth})$ . A later viewport scale converts these normalized coordinates into screen pixels.

## Part 3: Orthographic Projection and Wireframe Rendering

After transformation, each 3D vertex is projected orthographically by dropping the z-coordinate. The x and y values are then mapped to the framebuffer. The y-coordinate is inverted because framebuffer coordinates grow downward.

```
screen_x = viewport_center_x + point.x * SCREEN_MODEL_SCALE;
screen_y = viewport_center_y - point.y * SCREEN_MODEL_SCALE;
```

For every triangular face, the three projected vertices are connected using the Bresenham line algorithm implemented in Assignment 1. This produces a wireframe representation of the mesh.

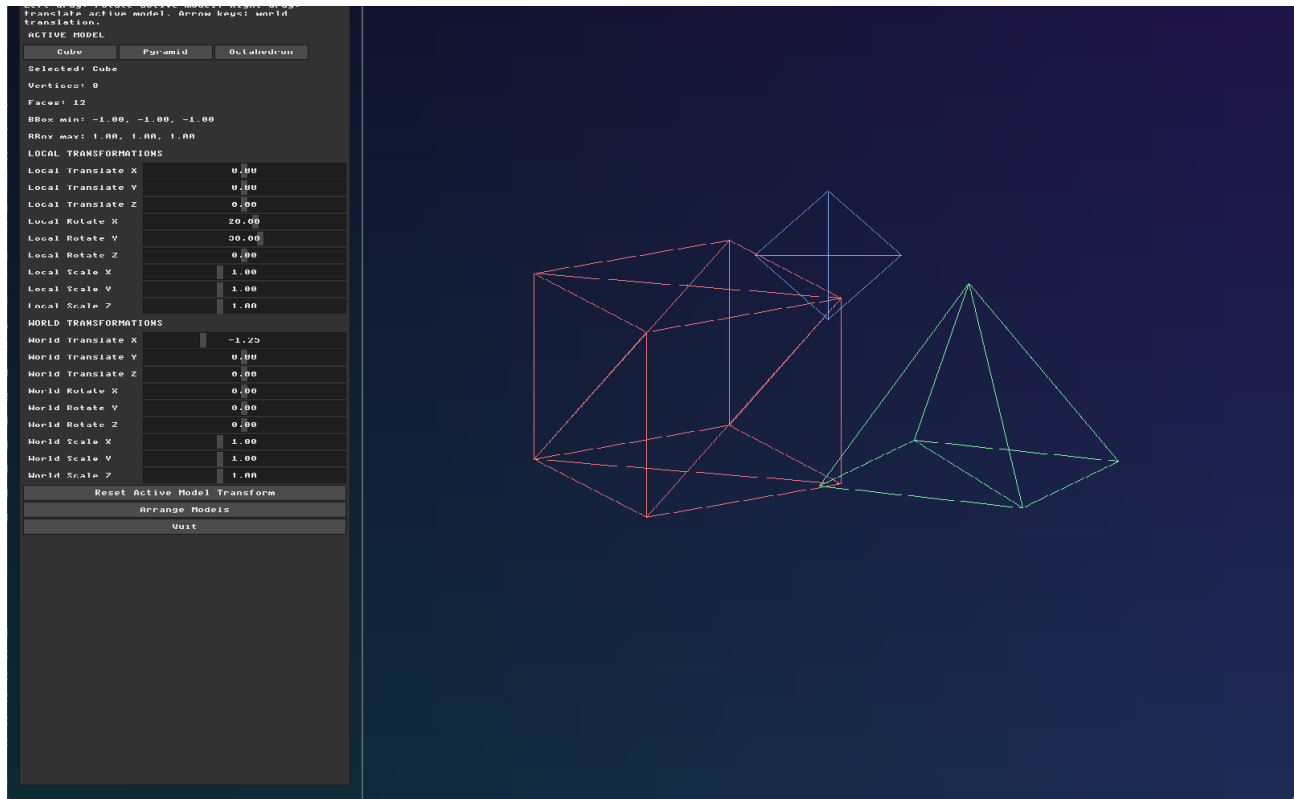


Figure 1. Multiple OBJ wireframe models rendered in the same scene with the transformation GUI.

## Part 4: Transformation Matrices and Immediate Mode GUI

Each model stores an independent transformation state. The GUI contains sliders for all three axes of local and world translation, rotation, and scale.

```
struct TransformState {
    glm::vec3 local_translation;
    glm::vec3 local_rotation;
    glm::vec3 local_scale;
    glm::vec3 world_translation;
    glm::vec3 world_rotation;
    glm::vec3 world_scale;
};
```

Because MicroUI is an Immediate Mode GUI, the slider values are stored in external variables and read again every frame. Moving a slider immediately changes the transformation matrix and therefore the rendered model.

## Part 5: Applying Local and World Transformations

The final model matrix is assembled from normalization, local transformations, and world transformations. Matrix multiplication is evaluated from right to left:

$$M = \text{World} * \text{Local} * \text{Normalization}$$

The local matrix is constructed as local translation, local rotation, and local scale. The world matrix is constructed in the same way. Applying world transformations outside the local matrix makes it possible to distinguish movement relative to the model from movement relative to the scene origin.

Case A: the cube was translated in its local frame and then rotated around the world origin. The translated position is affected by the world rotation, producing an orbit-like result.

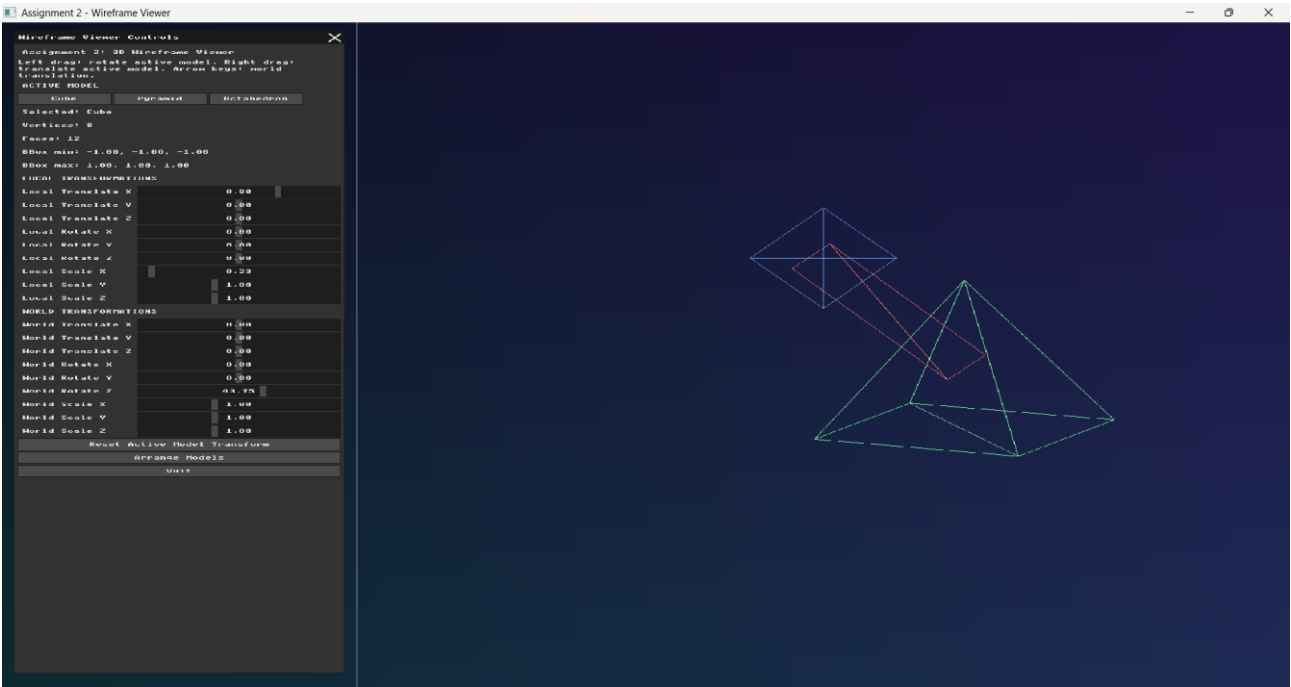


Figure 2. Local translation followed by world rotation.

Case B: the cube was translated in the world frame and rotated in its local frame. The object rotates around its own local center and is then positioned in the scene.

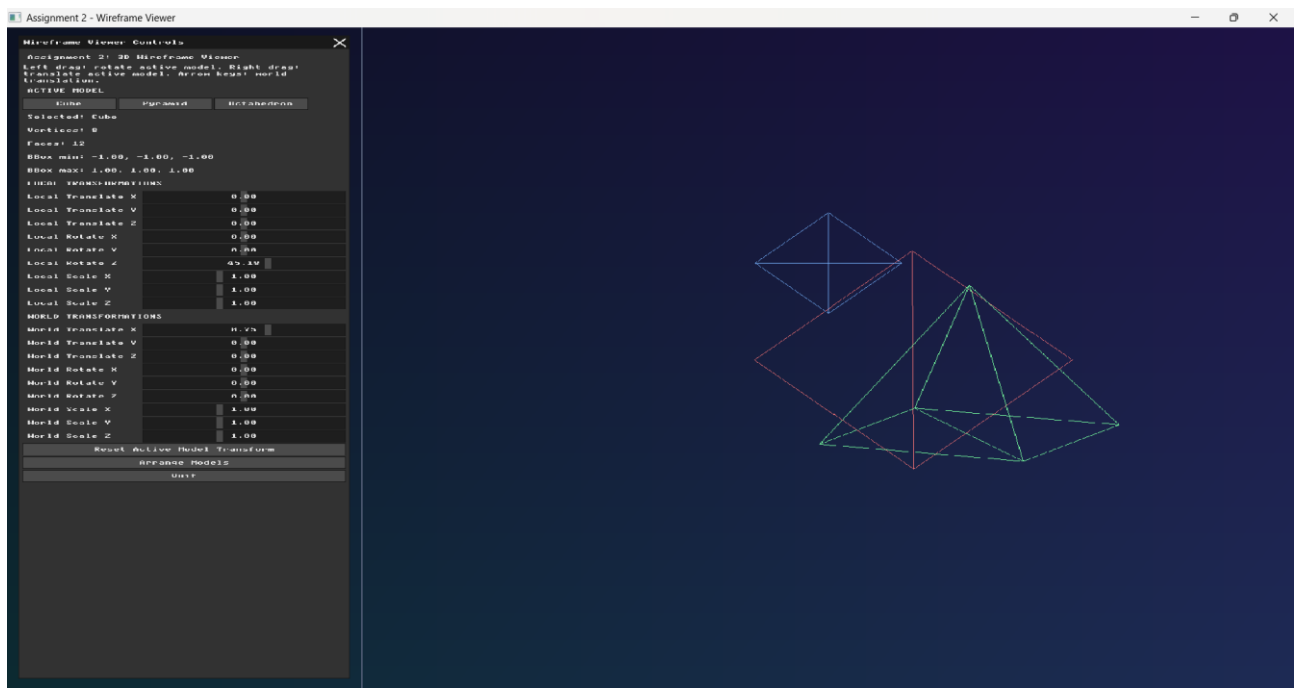


Figure 3. World translation followed by local rotation.

## Part 6: Interactive Input Modifiers

Direct manipulation was implemented in addition to the GUI controls. The arrow keys modify the active model's world translation. This provides a simple keyboard-based movement method.

- Left Arrow / Right Arrow: decrease or increase World Translation X.
- Up Arrow / Down Arrow: increase or decrease World Translation Y.

The input modifies the same transformation state used by the GUI, so the sliders and rendered model remain synchronized.

## Part 7: Pair Programming Extensions

### 7.1 Multiple Model Management

The application loads and stores three different OBJ meshes simultaneously. Each mesh contains its own vertices, faces, bounding box, color, and transformation state. Buttons in the GUI select the active model.

```
std::vector<Mesh> meshes;
int active_model = 0;
Mesh &selected = meshes[active_model];
```

Only the active model is affected by the transformation sliders, keyboard controls, and mouse controls. The models can therefore be arranged independently to compose a scene.

### 7.2 Advanced Mouse Control

Two different mouse interaction methods were implemented:

- Left-button drag rotates the active model around its local X and Y axes.
- Right-button drag translates the active model in world X and Y.

The application records the previous mouse position and converts the difference between frames into rotation angles or translation distances. Direct manipulation is enabled only inside the scene area, preventing conflicts with the GUI.

```
delta_x = current_mouse_x - previous_mouse_x;  
delta_y = current_mouse_y - previous_mouse_y;
```

## Testing and Results

The following features were tested successfully:

- GLM integration and project configuration.
- OBJ loading for cube, pyramid, and octahedron models.
- Correct vertex and face counts in the GUI.
- Bounding-box calculation and uniform normalization.
- Orthographic projection and Bresenham wireframe rendering.
- Independent local and world translation, rotation, and scale.
- Active-model selection and independent scene composition.
- Keyboard translation, left-drag rotation, and right-drag translation.
- Reset and Arrange Models controls.

## Conclusion

This assignment extended the framebuffer application from two-dimensional drawing to interactive three-dimensional geometry. The project loads OBJ meshes, normalizes arbitrary coordinates, projects transformed vertices orthographically, and renders triangle edges as wireframes using Bresenham's algorithm. GLM matrices support independent local and world transformations, while the Immediate Mode GUI, keyboard, and mouse provide real-time control. The pair-programming extensions add multiple independently controlled models and two intuitive mouse interaction methods.